

Indian Institute of Information Technology Bhagalpur

(An Institute of National Importance Under Act of Parliament)



COMPILER DESIGN - CS312

Submitted by:
Priyanshu (2201124CS)
Class: B. Tech. – III
Semester: VI

Course Instructor: **Dr. Ujjwal Biswas**
Junior Technical Superintendent: **Mr. Luv Pandey**

**Department of Computer Science & Engineering IIIT
BHAGALPUR, BIHAR 813210, INDIA**

Jan-May 2025

Contents

1. Introduction to Compiling:-

1.1 What is a Compiler?	-----3
1.2 Types of Compiler	-----4
1.3 Analysis of the Source Program	-----5
1.4 Overview of The Translation Process of a Source Program	-----6
1.5 Differences Between Compiler and Interpreter	-----7
1.6 The Phases of the Compiler	-----8

2. Compiler Core Parts and Tools:-

2.1 Cousins of The Compiler	-----9
2.2 Grouping of Phases for the Compiler	-----10
2.3 Front-End and Back-End of Compiler	-----11
2.4 Pass Structure in Details	-----11
2.5 Compiler Construction Tools	-----12

3. Core Design Part of the Compiler:-

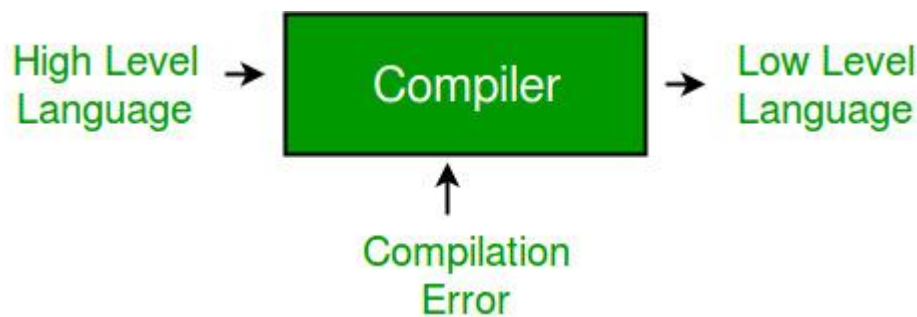
3.1 Simple One-Pass Compiler	-----12
3.2 Overview of Syntax Definition	-----13
3.3 Syntax Directed Translation,	-----15
3.4 What is Parsing in Compiler Design?	
3.5 Why Parsing is Important in Compiler Design?	-----17
3.6 What are Symbol Tables?	-----19
3.7 What is the Function of a Symbol Table in Compiler Design?	-----24

1. Introduction to Compiling:-

Programming languages are used to describe computations for both people and machines. Software relies on programming languages, which must be translated into a form executable by a computer through compilers. This book focuses on compiler design and implementation, showing how a few core ideas apply to many languages and machines. Compiler design touches on programming languages, machine architecture, algorithms, and software engineering. The chapter introduces different types of language translators, the structure of a typical compiler, trends in languages and architecture, and the broader applications of compiler technology. It also outlines essential programming-language concepts for studying compilers.

1.1 What is a Compiler?

Simply stated, a compiler is a program that can read a program in one language | the source language | and translate it into an equivalent program in another language | the target language; see Fig. 1.1. An important role of the compiler is to report any errors in the source program that it detects during the translation process.



1.2 Types of Compiler

Compilers can be categorized into several types based on various factors, such as their functionality, the target machine, and the compilation process. Here are the main types of compilers:

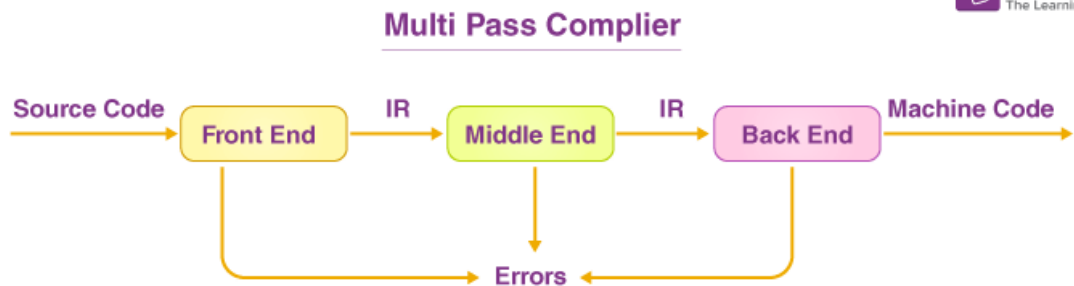
1. Single-pass Compiler:

A single-pass compiler processes the source code in one pass (or scan) from top to bottom. It generates the object code directly as it reads the

source program, making it fast but less efficient for large programs due to limited analysis and optimization.

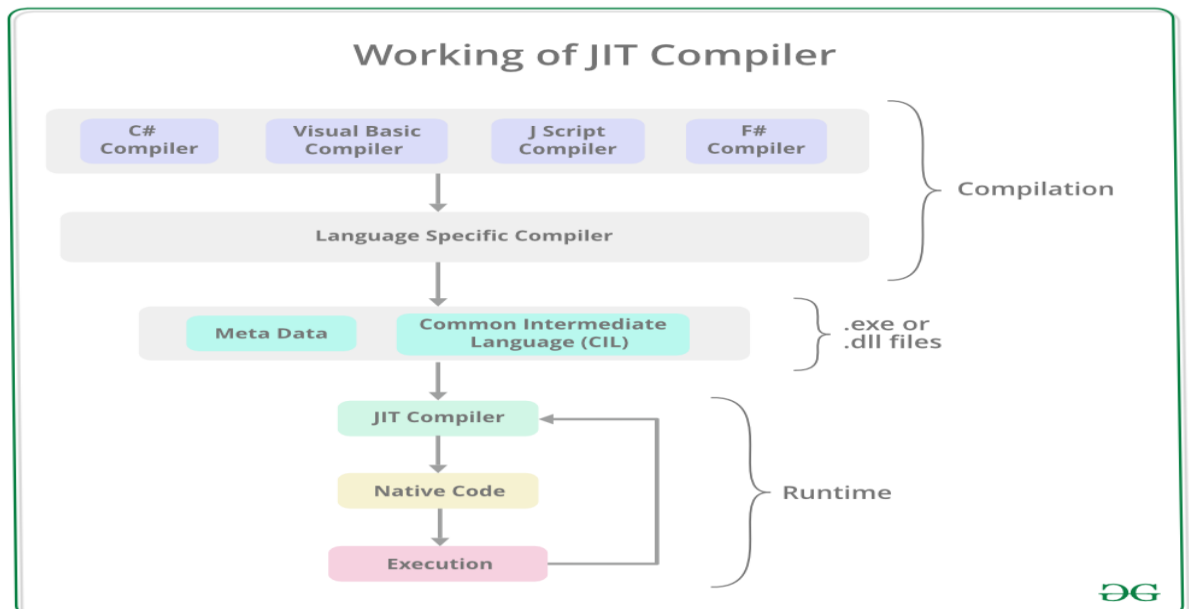
2. Multi-pass Compiler:

A multi-pass compiler processes the source code in multiple passes. In each pass, different aspects of the code are analyzed, optimized, and transformed into intermediate or final code. This approach allows for better optimizations and more thorough analysis than single-pass compilers.



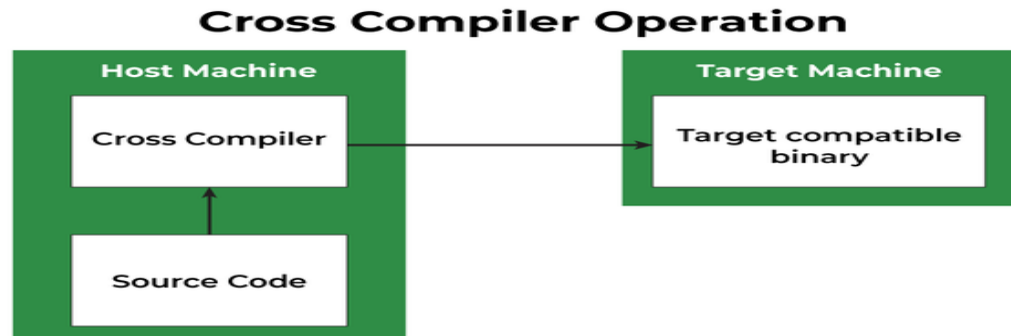
3. Just-in-time (JIT) Compiler:

JIT compilers translate high-level code into machine code at runtime, just before execution. This allows programs to be optimized based on the current execution context, improving performance for dynamic applications (e.g., Java or C# programs).



4. Cross Compiler:

A cross compiler generates code for a different target platform than the one on which it is running. For example, a compiler on a Windows machine that generates code for an embedded system.



5. Retargetable Compiler:

A retargetable compiler can generate code for different target machines by reconfiguring parts of the compiler, allowing for easier adaptation to different hardware platforms.

6. Optimizing Compiler:

An optimizing compiler improves the efficiency of the generated code in terms of execution speed, memory usage, or both. It performs various optimization techniques during the compilation process, such as loop unrolling and dead code elimination.

7. Incremental Compiler:

An incremental compiler recompiles only the parts of the program that have changed, rather than recompiling the entire source code. This speeds up the development cycle and is especially useful in large projects.

8. Interpreting Compiler (or Interpreter):

Although technically not a compiler, an interpreter directly executes instructions written in a high-level language without translating them into machine code. It reads and executes the program line by line.

9. Bytecode Compiler:

A bytecode compiler generates an intermediate code, called bytecode, which is platform-independent and can be executed by a virtual machine (e.g., the Java Virtual Machine (JVM) or the Python interpreter).

Each type of compiler has its strengths and use cases depending on the performance needs, target platforms, and language features.

1.3 Analysis of the Source Program

The analysis phase of the compiler involves breaking down the source program into components that can be processed by the later stages of the compiler. The key steps involved in analysis are:

- **Lexical Analysis (Scanning):** This phase divides the source code into tokens (the smallest units of meaning in the language, such as keywords, operators, and identifiers). The lexical analyzer reads the source program character by character and groups them into these tokens, which are then passed to the next phase.
- **Syntax Analysis (Parsing):** In this phase, the sequence of tokens produced by the lexical analyzer is checked against the grammatical structure of the programming language. A syntax tree (or parse tree) is generated that reflects the syntactic structure of the source code according to the language's grammar.
- **Semantic Analysis:** After the syntax is validated, the semantic analysis phase checks for the logical consistency of the program. This includes type checking, ensuring that variables are declared before use, and verifying that operations are performed on compatible types. It ensures that the program follows the rules and constraints of the language's semantics.

1.4 Overview of The Translation Process of a Source Program

The translation process involves transforming the source program into executable machine code. This is achieved in a series of steps:

1. **Lexical Analysis:** The source code is converted into a stream of tokens, which are the smallest meaningful units in the language.

2. **Syntax Analysis:** The tokens are parsed to build a syntax tree that represents the grammatical structure of the program.
3. **Semantic Analysis:** This phase checks the program for semantic errors, ensuring that the program adheres to the logical rules of the language (e.g., type correctness, variable declarations).
4. **Intermediate Code Generation:** The compiler generates an intermediate representation of the source code. This code is often platform-independent and allows for further optimization.
5. **Code Optimization:** The intermediate code is optimized to improve performance, reduce memory usage, and enhance execution efficiency.
6. **Code Generation:** The optimized intermediate code is translated into machine code or assembly code specific to the target architecture.
7. **Code Linking and Assembly:** The machine code is linked with libraries and other modules, and the final executable program is produced.

1.5 Differences Between Compiler and Interpreter

COMPARISON	COMPILER	INTERPRETER
Input	It takes an entire program at a time.	It takes a single line of code or instruction at a time.
Output	It generates intermediate object code.	It does not produce any intermediate object code.
Working mechanism	The compilation is done before execution.	Compilation and execution take place simultaneously.
Speed	Comparatively faster	Slower
Memory	Memory requirement is more due to the creation of object code.	It requires less memory as it does not create intermediate object code.
Errors	Display all errors after compilation, all at the same time.	Displays error of each line one by one.
Error detection	Difficult	Easier comparatively
Pertaining Programming languages	C, C++, C#, Scala, typescript uses compiler.	PHP, Perl, Python, Ruby uses an interpreter.

1.6 The Phases of the Compiler

The process of compiling a program is divided into several distinct phases, each serving a specific function. These phases can be grouped into two main stages: the analysis and synthesis stages.

1. Lexical Analysis (Scanning):

This phase breaks the source code into tokens, which are basic language elements like keywords, identifiers, literals, and operators.

2. Syntax Analysis (Parsing):

Here, the token stream is parsed according to the syntax rules of the programming language. The output is a syntax tree that represents the structure of the program.

3. Semantic Analysis:

This phase ensures that the syntax of the program makes sense semantically. It checks for type errors, uninitialized variables, and other logic errors, and annotates the syntax tree with necessary information like types and scopes.

4. Intermediate Code Generation:

The compiler generates an intermediate representation (IR) of the source code. The IR is often platform-independent, making it easier to optimize and adapt for different architectures.

5. Code Optimization:

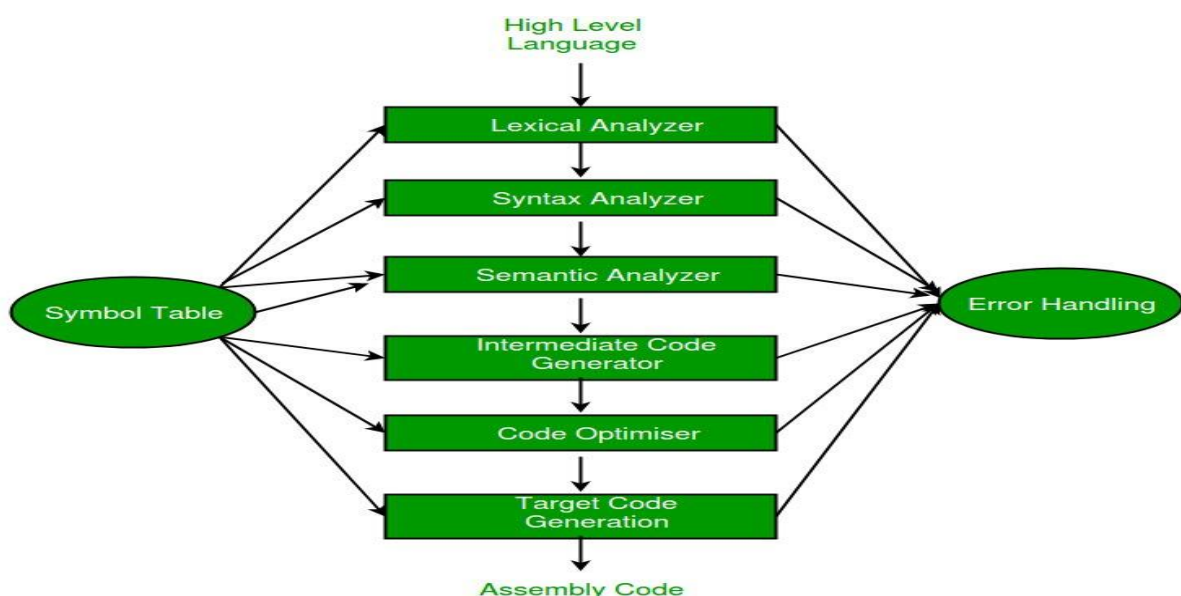
In this phase, the intermediate code is optimized for efficiency. This could involve eliminating redundant code, reducing memory usage, or improving execution speed. The goal is to produce a more efficient intermediate representation.

6. Code Generation:

The intermediate code is translated into the target machine code or assembly code, which is specific to the target architecture.

7. Code Linking and Assembly:

The final machine code is linked with libraries and other modules, and it is assembled into an executable file that can be run on the target machine.

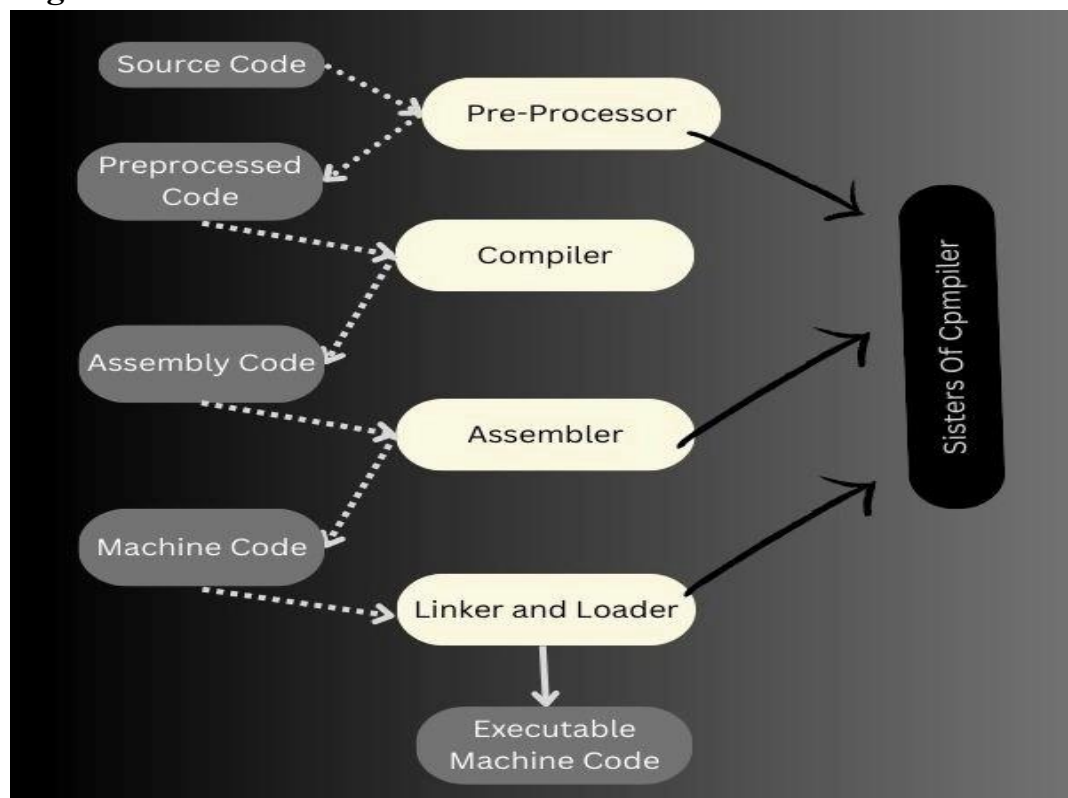


2. Compiler Core Parts and Tools:-

2.1 Cousins of the Compiler

The compiler is part of a family of tools essential for software development. Its cousins include:

- **Preprocessors:** Perform initial processing of source code (e.g., handling macros and includes in C/C++).
- **Assemblers:** Convert assembly language into machine code.
- **Linkers:** Combine multiple object files into a single executable, resolving references among them.
- **Loaders:** Load the executable file into memory for execution. These tools work together to transform high-level source code into executable binaries.
- **Diagram:-**

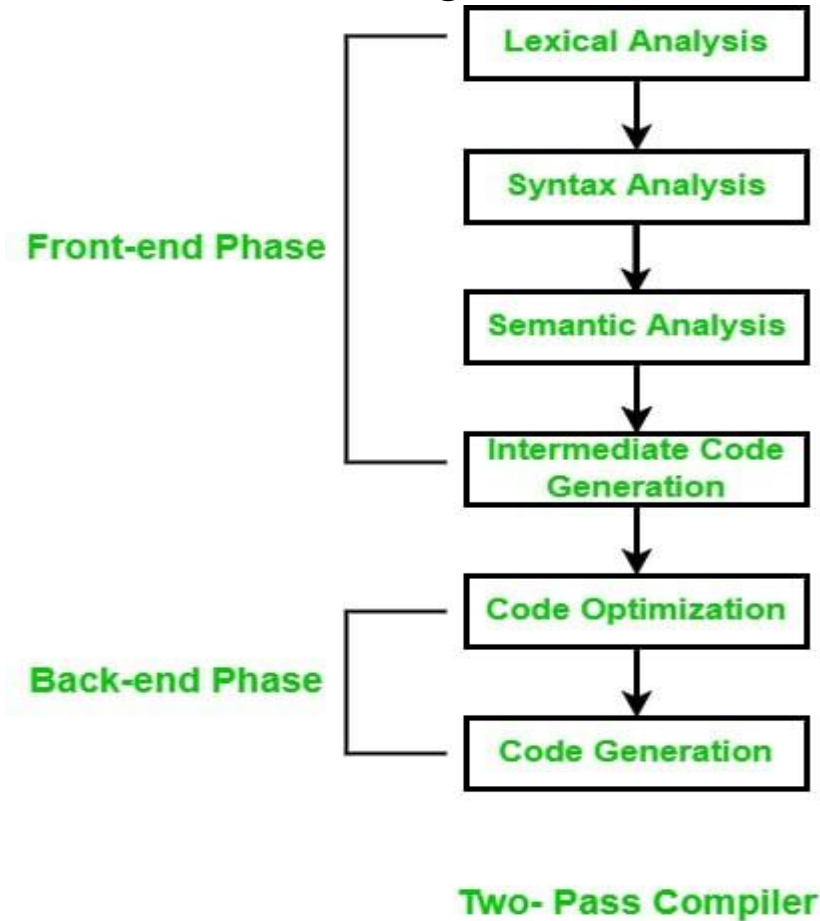


2.2 Grouping of Phases for the Compiler

The compilation process is divided into six primary phases, grouped as:

- **Analysis Phase:** Converts source code into an intermediate representation (IR).
 1. Lexical Analysis: Tokenizing the input source code.

2. Syntax Analysis: Ensures grammatical structure (parsing).
3. Semantic Analysis: Validates semantics, such as type checking.
- **Synthesis Phase:** Transforms IR into target code.
4. Intermediate Code Generation: Creates a language-independent IR.
5. Code Optimization: Enhances performance by refining the IR.
6. Code Generation: Produces the final machine code.
- **Diagram:-**



2.3 Front-End and Back-End of Compiler

- **Front-End:** Includes the analysis phases. It handles language-specific tasks such as:
 - Lexical, syntax, and semantic analysis.
 - Generating IR.
 - Error detection and reporting.
- **Back-End:** Includes the synthesis phases. It is responsible for:
 - Optimizing the IR for performance.
 - Translating IR into target machine code.
 - Managing resource allocation, such as registers and memory.

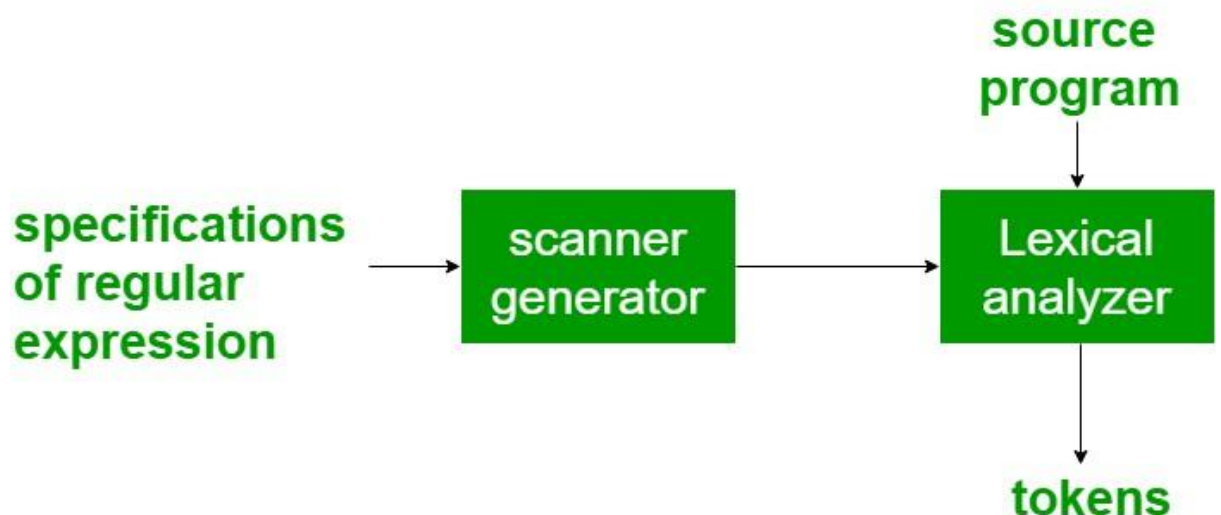
2.4 Pass Structure in Detail

- **Single-Pass Compiler:** Completes the compilation in one go without revisiting the source code or IR. Suitable for simple languages.
- **Multi-Pass Compiler:** Revisits the code or IR multiple times for complex analysis and optimizations. Each pass performs a specific set of tasks:
 - First Pass: Analysis phase, including lexical and syntax analysis.
 - Intermediate Pass(es): Optimization and transformations.
 - Final Pass: Code generation and machine-specific optimizations.

2.5 Compiler Construction Tools

These tools aid in building efficient and robust compilers:

- Lexical Analyzers (Lex/Flex): Automate token generation from source code.
- Parser Generators (Yacc/Bison): Create parsers for syntax analysis.
- Intermediate Code Generators: Translate source into IR.
- Code Optimizers: Improve performance by optimizing IR or final code.
- Code Generators: Convert IR into target machine code.
- Debuggers and Profilers: Help debug and analyze the performance of compiled code.
- Diagram:-



3. Core Design Part of the Compiler :-

3.1 Single One-pass Compiler

INTRODUCTION: In computer programming, a one-pass compiler is a compiler that passes through the parts of each compilation unit only once, immediately translating each part into its final machine code. This is in contrast to a multi-pass compiler which converts the program into one or more intermediate representations in steps between source code and machine code, and which reprocesses the entire compilation unit in each sequential pass.

OVERVIEW

- **Language Definition**

1. **Appearance of programming language**

Vocabulary : Regular expression Syntax : Backus-Naur Form(BNF) or Context Free Form(CFG)

2. **Semantics : Informal language or some examples**

-

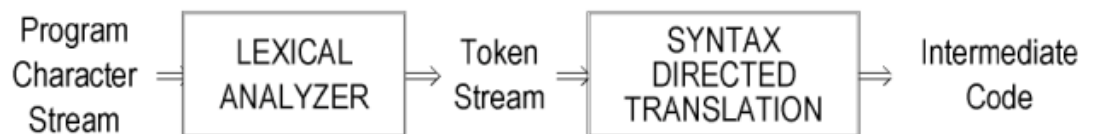


Fig 2.1. Structure of our compiler front end

3.2 SYNTAX DEFINITION

- To specify the syntax of a language : CFG and BNF

- ❖ Example : if-else statement in C has the form of statement $\rightarrow$ if (expression) statement else statement
- An alphabet of a language is a set of symbols.

- ❖ Examples : $\{0,1\}$ for a binary number system (language) $= \{0,1,100,101,\dots\}$ $\{a,b,c\}$ for language $= \{a,b,c, ac,abcc..\}$ $\{if,(,),else \dots\}$ for a if statements $= \{if(a==1)goto 10, if--\}$

- A string over an alphabet

- ❖ is a sequence of zero or more symbols from the alphabet.

- ❖ Examples : 0,1,10,00,11,111,0202 ... strings for a alphabet $\{0,1\}$
- ❖ Null string is a string which does not have any symbol of the alphabet.
- Language
 - ❖ Is a subset of all the strings over a given alphabet.
 - ❖ Alphabets A_i Languages L_i for A_i $A_0 = \{0,1\}$

$$L_0 = \{0,1,100,101,\dots\}$$

$$A_1 = \{a,b,c\} \quad L_1 = \{a,b,c, ac, abcc..\}$$
- $A_2 = \{\text{all of C tokens}\} \quad L_2 = \{\text{all sentences of C program}\}$
- Example 2.1. Grammar for expressions consisting of digits and plus and minus signs.
 - ❖ Language of expressions $L = \{9-5+2, 3-1, \dots\}$.
 - ❖ The productions of grammar for this language L are:

$$\text{list} \rightarrow \text{list} + \text{digit} \quad \text{list} \rightarrow$$

$$\text{list} - \text{digit} \quad \text{list} \rightarrow \text{digit}$$

$$\text{digit} \rightarrow 0|1|2|3|4|5|6|7|8|9$$
 - ❖ list, digit : Grammar variables, Grammar symbols
 - ❖ 0,1,2,3,4,5,6,7,8,9,-,+: Tokens, Terminal symbols
- Convention specifying grammar
 - ❖ Terminal symbols : boldface string if, num, id
 - ❖ Nonterminal symbol, grammar symbol : italicized names, list, digit ,A,B
- Grammar $G = (N, T, P, S)$
 - ❖ N : a set of nonterminal symbols
 - ❖ T : a set of terminal symbols, tokens
 - ❖ P : a set of production rules S : a start symbol, $S \in N$
- Grammar G for a language $L = \{9-5+2, 3-1, \dots\}$
 - ❖ $G = (N, T, P, S)$

$$N = \{\text{list}, \text{digit}\}$$

$T = \{0,1,2,3,4,5,6,7,8,9,-,+\}$ P

$: \text{list} \rightarrow \text{list} + \text{digit} \quad \text{list} \rightarrow \text{list} -$

$\text{digit} \quad \text{list} \rightarrow \text{digit} \quad \text{digit} \rightarrow$

$0|1|2|3|4|5|6|7|8|9 \quad S = \text{list}$

- Some definitions for a language L and its grammar G
- Derivation : A sequence of replacements $S \Rightarrow \alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n$ is a derivation of α_n . Example, A derivation $1+9$ from the grammar G
- leftmost derivation $\text{list} \Rightarrow \text{list} + \text{digit} \Rightarrow \text{digit} + \text{digit} \Rightarrow 1 + \text{digit} \Rightarrow 1 + 9$
- right most derivation $\text{list} \Rightarrow \text{list} + \text{digit} \Rightarrow \text{list} + 9 \Rightarrow \text{digit} + 9 \Rightarrow 1 + 9$
- Language of grammar $L(G)$ $L(G)$ is a set of sentences that can be generated from the grammar G . $L(G) = \{x \mid S \Rightarrow^* x\}$ where $x \in$ a sequence of terminal symbols
- Example: Consider a grammar $G = (N, T, P, S)$: $N = \{S\}$ $T = \{a, b\}$ $S = S$ $P = \{S \rightarrow aSb \mid \epsilon\}$
- Is $aabb$ a sentence of $L(G)$? (derivation of string $aabb$)
 $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aa\epsilon bb \Rightarrow aabb$ (or $S \Rightarrow^* aabb$) so, $aabb \in L(G)$
- there is no derivation for aa , so $aa \notin L(G)$
- note $L(G) = \{a^n b^n \mid n \geq 0\}$ where $a^n b^n$ means n a's followed by n b's.

3.3 SYNTAX DIRECTED TRANSLATION

In syntax directed translation, along with the grammar we associate some informal notations and these notations are called semantic rules.

So we can say that

Grammar + semantic rule = SDT (syntax directed translation)

- In syntax directed translation, every non-terminal can get one or more than one attribute or sometimes 0 attribute depending on the type of the attribute. The value of these attributes is evaluated by the semantic rules associated with the production rule.
- In the semantic rule, attribute is VAL and an attribute may hold anything like a string, a number, a memory location and a complex record
- In Syntax directed translation, whenever a construct is encountered in the programming language then it is translated according to the semantic rules defined in that particular programming language.

Example

Production	Semantic Rules
$E \rightarrow E + T$	$E.val := E.val + T.val$
$E \rightarrow T$	$E.val := T.val$
$T \rightarrow T * F$	$T.val := T.val * F.val$
$T \rightarrow F$	$T.val := F.val$
$F \rightarrow (F)$	$F.val := F.val$
$F \rightarrow \text{num}$	$F.val := \text{num.lexval}$

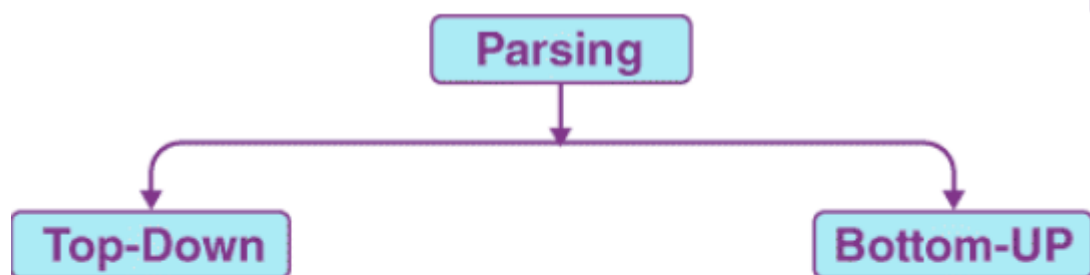
E.val is one of the attributes of E.

num.lexval is the attribute returned by the lexical analyzer.

3.4 What is Parsing in Compiler Design?

The process of transforming the data from one format to another is called Parsing. This process can be accomplished by the parser. The parser is a component of the translator that helps to organise linear text structure following the set of defined rules which is known as grammar.

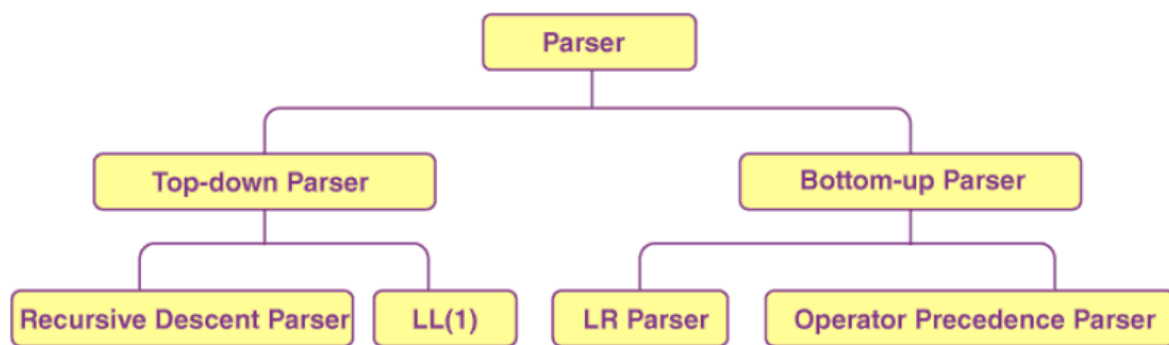
Types of Parsing:



There are two types of Parsing:

- The Top-down Parsing

- The Bottom-up Parsing

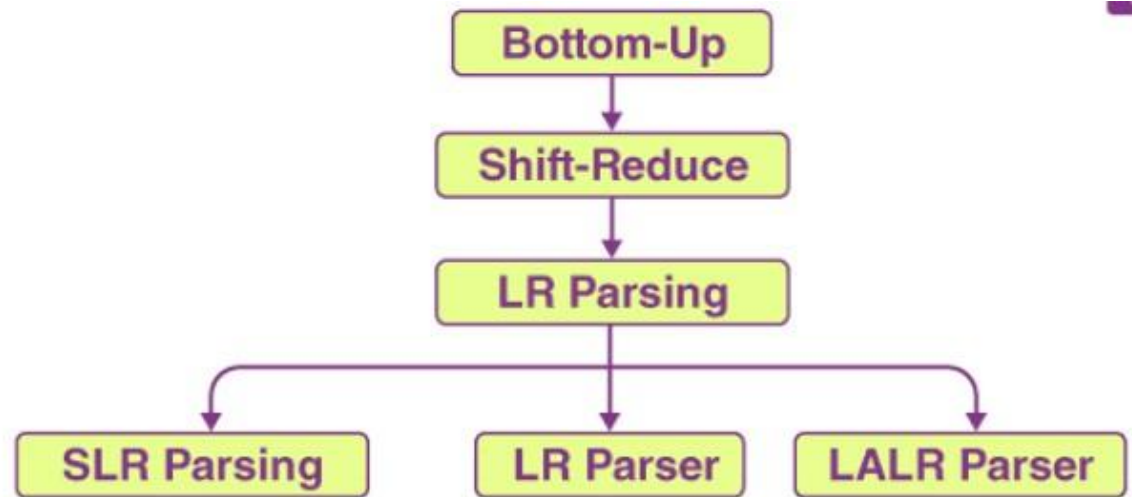


Top-down Parsing: When the parser generates a parse with top-down expansion to the first trace, the left-most derivation of input is called top-down parsing. The top-down parsing initiates with the start symbol and ends on the terminals. Such parsing is also known as predictive parsing.

Recursive Descent Parsing: Recursive descent parsing is a type of top-down parsing technique. This technique follows the process for every terminal and non-terminal entity. It reads the input from left to right and constructs the parse tree from right to left. As the technique works recursively, it is called recursive descent parsing.

Back-tracking: The parsing technique that starts from the initial pointer, the root node. If the derivation fails, then it restarts the process with different rules.

Bottom-up Parsing: The bottom-up parsing works just the reverse of the top-down parsing. It first traces the rightmost derivation of the input until it reaches the start symbol.



Shift-Reduce Parsing: Shift-reduce parsing works on two steps: Shift step and Reduce step.

Shift step: The shift step indicates the increment of the input pointer to the next input symbol that is shifted.

Reduce Step: When the parser has a complete grammar rule on the right-hand side and replaces it with RHS.

LR Parsing: LR parser is one of the most efficient syntax analysis techniques as it works with context-free grammar. In LR parsing L stands for the left to right tracing, and R stands for the right to left tracing.

3.5 Why is parsing useful in compiler designing?

In the world of software, every different entity has its criteria for the data to be processed. So parsing is the process that transforms the data in such a way so that it can be understood by any specific software.

The Technologies Use Parsers:

- The programming languages like Java.
- Database languages like SQL.
- The Scripting languages.
- The protocols like HTTP.
- The XML and HTML.

3.6 What is a symbol table?

A symbol table is an important Data Structure used by compilers to manage identifiers in a program. An identifier is a name given to a variable, function, class or other programming construct that is used to represent some data or functionality in a program. Identifiers must be declared before they are used in a program, and their properties (such as data type, scope, and memory location) must be known by the compiler to generate the correct code. A symbol table provides a way for the compiler to store this information and access it as needed during the compilation process.

A symbol table typically consists of a set of entries, each of which represents an identifier used in the program. Each entry contains information such as the identifier's name, data type, scope, memory location, and any other attributes that may be needed by the compiler. The symbol table is created and populated during the compilation phase, as the compiler scans the program for identifiers and their declarations. Once the symbol table is populated, it can be used by the compiler in several ways. For example, the compiler can use the symbol table to check for errors in the program, such as undeclared variables or type mismatches.

Implementation of Symbol table

If a compiler only needs to process a small quantity of data, the symbol table can be implemented as an unordered list, which is simple to code but only works for small tables. The following methods can be used to create a symbol table:

List

A List is a collection of elements in which each element has a position or an index. This is one of the ways to create a symbol table is to use a List to store the entries for each identifier. This method is simple and easy to implement, but it may not be efficient for large symbol tables because searching for a specific identifier requires iterating through the entire List.

Linked List

A Linked List is a data structure in which each element contains a reference to the next element in the list. One way to create a symbol table is to use a Linked List to store the entries for each identifier. This method can be more efficient than a List for searching because it allows for faster traversal of the entries. However, it can be slower than other methods for inserting or deleting entries because it requires modifying the references between elements.

Binary Search Tree

A Binary Search Tree is a data structure in which each node has at most two children, and the values in the left subtree are less than the value in the node, and the values in the right subtree are greater than the value in the node. One way to create a symbol table is to use a Binary Search Tree to store the entries for each identifier. This method is efficient for searching because it allows for a binary search to quickly find a specific identifier. However, it can be slower than other methods for inserting or deleting entries because it may require rebalancing the tree.

Hash Table

A Hash Table is a data structure that uses a hash function to map keys to values. One way to create a symbol table is to use a Hash Table to store the entries for each identifier. This method is efficient for searching, inserting, and deleting entries because it allows for constant-time access to entries. However, it may require more memory than other methods because it needs to store the hash table and handle collisions between entries.

Features Of Symbol Table

This symbol table is used for the following purposes:

- It is used to keep all of the names of all entities in one place in a structured format.
- It is used to check whether or not a variable has been declared.
- It is used to identify the scope of a name.
- It is used to implement type checking in source code by ensuring if assignments and expressions are semantically accurate or not.
- It is utilized in the compiler's different phases, as shown below:
 - Lexical Analysis:** New table entries are created in the table, For example, entries about tokens.
 - Syntax Analysis:** Adds the information about attribute type,, dimension, scope line of reference, use, etc in the table.
 - Semantic Analysis:** Checks for semantics in the table, i.e., verifies that expressions and assignments are semantically accurate (type checking) and updates the table appropriately.
 - Intermediate Code generation:** The symbol table is used to determine how much and what type of run-time is allocated, as well as to add temporary variable data.

Code Optimization: Uses information from the symbol table for machine-dependent optimization.

Target Code generation: Uses the address information of the identifier in the table to generate code.

Symbol Table entries in Compiler Design

Each item in the symbol table is linked to a set of properties that assist the compiler at various stages.

Items stored in Symbol table:

- Labels in source languages
- Procedure and function names
- Variable names and constants
- Compiler generated temporaries
- Literal constants and strings

Information used by the compiler from the Symbol table:

- Data type and name
- Offset in storage
- Declaring procedures
- For parameters, whether parameter passing by value or by reference
- Number and type of arguments passed to function
- Base Address

Basic Operations of Symbol Table in Compiler Design

The core operations of a symbol table are Allocate, free, insert, lookup, set attribute, and get attribute. The allocation operation creates an empty symbol table. The free operation is used to remove all records and free the storage of a symbol table. As the name implies, the insert operation puts a name into a symbol table and returns a pointer to its entry. The lookup function looks up a

name and returns a reference to the corresponding entry. The set and get attributes associate an attribute with a given entry and get an attribute associated with a provided. Other steps may be introduced depending upon the requirements. For example, a delete action deletes a previously entered name.

Operations Of Symbol Table

Operations	Functions
allocate	To allocate a new empty symbol table
free	To remove all entries and free storage of symbol table
lookup	To search for a name and return a pointer to its entry
insert	To insert a name in a symbol table and return a pointer to its entry
set_attribute	To associate an attribute with a given array
get_attribute	To get an attribute associated with a given array

Information used by the compiler from Symbol table:

Symbol Identification

The symbol table enables the compiler to apprehend and keep track of variables, functions, and different symbols in your application.

Data Type Information

It stores data about the data types of variables, like whether they're numbers, text, or something else.

Variable Scope

It is information wherein variables are declared and where they may be used within your code.

Function Details

For functions, it stores their names, parameters, and what they do.

Error Detection

The symbol desk aids in detecting mistakes for your code by way of making sure that variables and functions are used correctly and consistently.

Code Generation

It assists in generating machine code or executable programs primarily based on the statistics saved, making your code run effectively.

3.7 What is the Function of a Symbol Table in Compiler Design?

The symbol table serves several functions:

- **Scope Management:** Keeps track of variable scopes to manage visibility and lifetime.
- **Type Checking:** Assists in verifying that operations on variables are semantically correct.
- **Memory Allocation:** Helps allocate storage for variables and functions.
- **Identifier Resolution:** Resolves references to identifiers during code generation.

REFERENCES

Book: *"Compilers: Principles, Techniques, and Tools" by Aho, Lam, Sethi, and Ullman (Dragon Book).* :- [Link](#)

TutorialsPoint: *Compiler Design Overview* :- [Link](#)

GeeksforGeeks: *Compiler Design* :- [Link](#)

Online Resources: *Coursera and edX courses on Compiler Construction.*

GNU Tools: *Documentation for Lex, Yacc, GCC, and related tools.*